المحاضره الأولى

History of C++

- C++, as the name implies, is essentially based on the C programming language.
- The C programming language was devised in the early 1970s at Bell Laboratories by Dennis Ritchie.
- It was designed as a system implementation language for the Unix operating system.
- The history of C and Unix are closely intertwined

- For this reason a lot of Unix programming is done with C.
- To some extent, C was originally based on the typeless language BCPL; however it grew well beyond that.

C++ Fundamentals

- How A computer thinks?

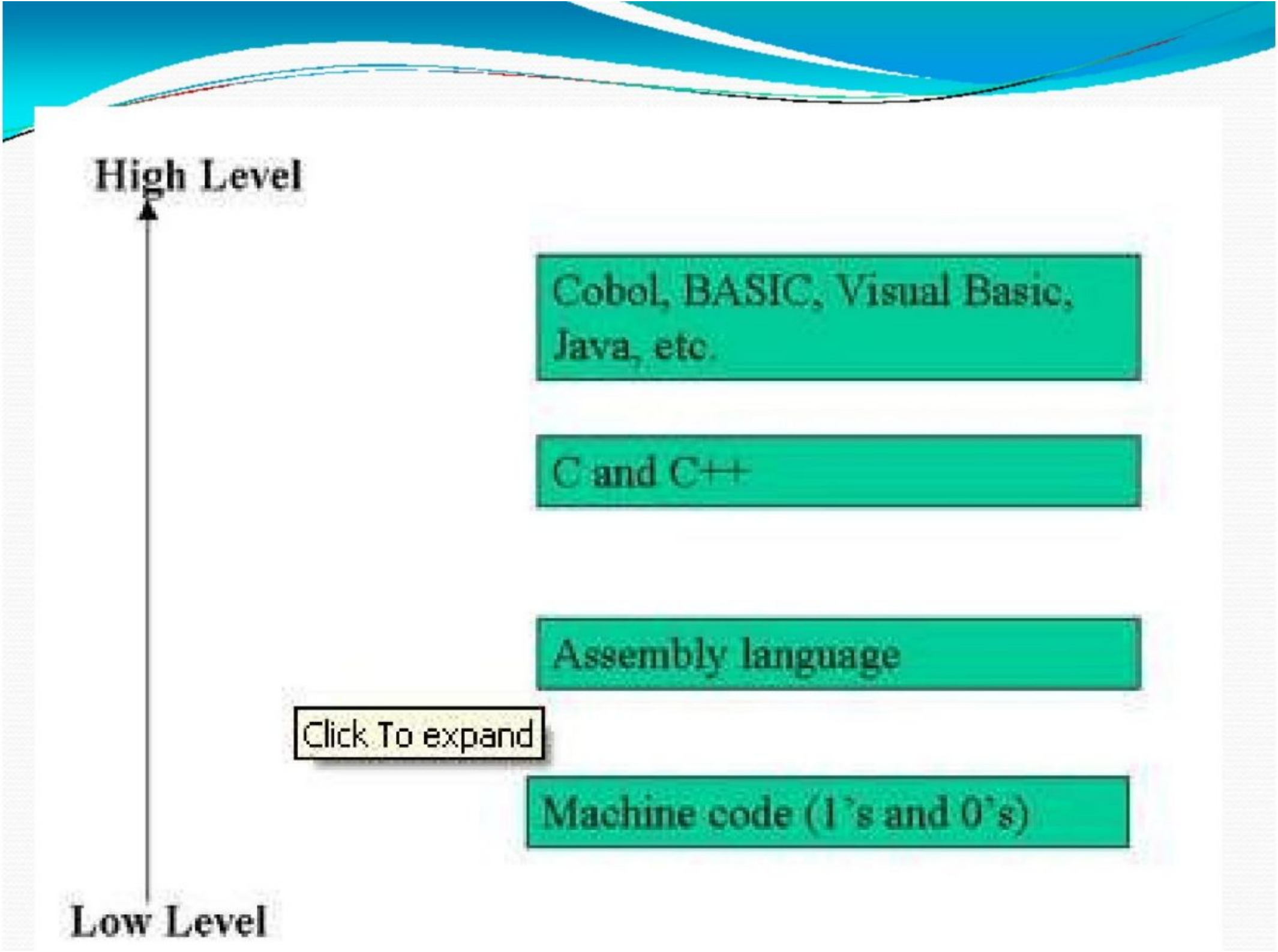
A computer thinks in 1's and 0's. Various switches are either on (1's), or off (0's).

- How a human thinks?

A *programming language* is a language that provides a bridge between the computer and human beings.

- A “**low-level” language** is one that is closer to a computer’s thinking than to a human language.
- Example Assembly language.

A “**high-level”language** is one that is closer to human language


High Level
Cobol, BASIC, Visual Basic, Java, etc.
C and C++
Assembly language
Click To expand
Machine code (1's and 0's)
Low Level

- **That data might be information about**

1. employees,
2. parts to a mathematical computation,
3. scientific data,
4. or even the elements of a game.

No matter what programming language or techniques you use,

What is the goals of programming

1. store,
2. manipulate,
3. and retrieve data.

Why we need to used variables ?

Data must be temporarily stored in the program, in order to be manipulated. This is accomplished via variables

What is the variables ?

A variable is simply a place in memory set aside to hold data of a particular type.

It is a specific section of the computer's memory that has been reserved to hold some data.

Why it is called variable ?

- It is called a variable because its value or content can vary.
- When you create a variable, you are actually setting aside a small piece of memory for storage purposes.
- The name you give the variable is actually a label for that address in memory.

- example, you might declare a variable in the following manner.
- **int j;**
- Then, you have **just allocated** four bytes of memory;
- you are using the variable j to refer to those four bytes of memory.
- You are also stating that the only type of data that j will hold, is whole numbers.
- (The int is a data type that refers to integers, or whole numbers.)
- Now, whenever you reference j in your code, you are actually referencing the contents being stored at a specific address in memory.

Getting Ready to Program

- Programs are written to instruct machines to carry out specific tasks or to solve specific problems.
- *Algorithm ?*
- Is a step-by-step procedure that accomplishes a desired task.
- Thus, programming is the activity of communicating algorithms to computers.
- The programming process is analogous, except that machines have no tolerance for ambiguity and must have all steps specified in a precise language and in tedious detail.

The Programming Process

- 1. Specify the task.
- 2. Discover an algorithm for its solution.
- 3. Code the algorithm in C++.
- 4. Test the code

- A computer is a digital electronic machine composed of three main components:

1. **processor,**
2. **memory,**
3. **input/output devices.**

- The processor is also called the *central processing unit*, or *CPU*.
- The processor carries out instructions that are stored in the memory.
- Along with the instructions, data also is stored in memory.
- The processor typically is instructed to manipulate the data in some desired fashion.

- . Input/output devices take information from agents external to the machine and provide information to those agents.

- **. Input devices are typically**

1. terminal keyboards,
2. disk drives,
3. and tape drives.

Output devices are typically

1. terminal screens,
2. printers,
3. disk drives,
4. and tape drives.

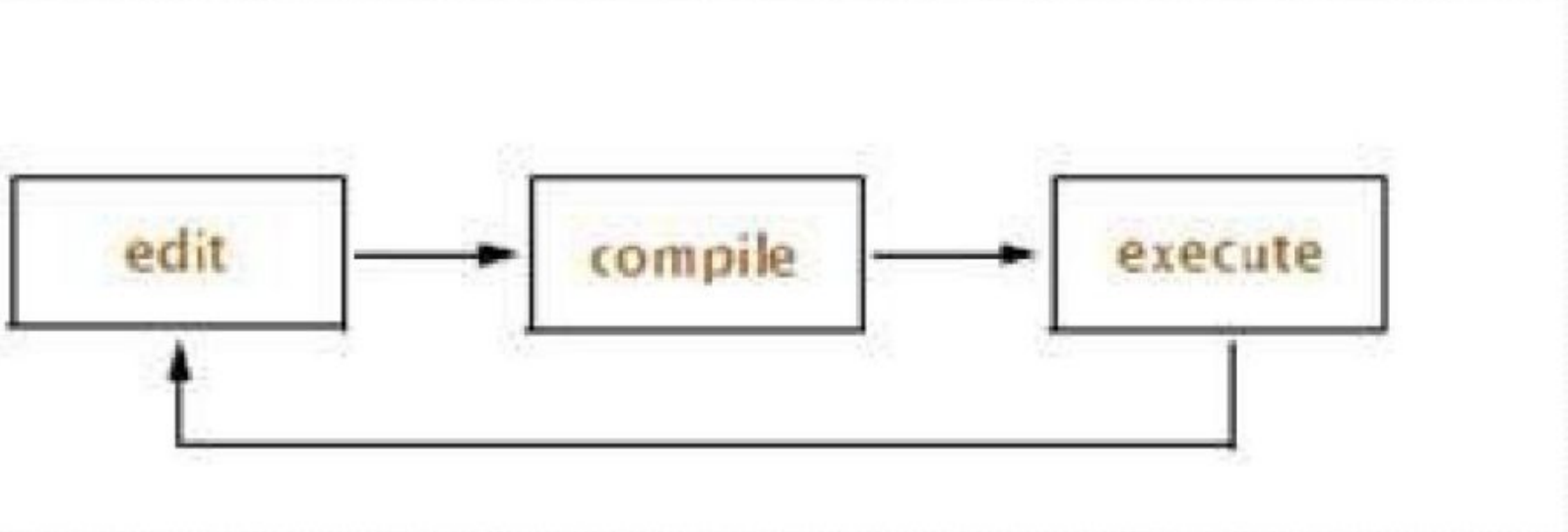
The physical makeup of a machine can be quite complicated, but the user need not be concerned with the details.

- When a simple program is compiled , three separate actions occur:

First the preprocessor is invoked, then the compiler, and finally the linker.

- The preprocessor modifies a copy of the source code by including other files and by making other changes.
- The compiler translates this into *object code*, which the linker then uses to produce the final executable file.
- A file that contains object code is called an *object file*.
- Object files, unlike source files, usually are not read by humans.

- When we speak of compiling a program, we really mean invoking the preprocessor, the compiler, and the linker.
- For a simple program, this is all done with a single command.
- After the programmer writes a program, it has to be compiled and tested.
- If modifications are needed, the source code has to be edited again.
- Thus, part of the programming process consists of this cycle:
- When the programmer is satisfied with the program performance, the cycle


edit
compile
execute

A First Program

- A first task for anyone learning to program is to print on the screen.
- Let us begin by writing the traditional first C++ program which prints the phrase *Hello, world!* On the screen. The complete program is

- // my first program in C++
- #include <iostream>
- int main ()
- {
- ";!cout << "Hello World
- return 0;
- }
- Hello World!

- **// my first program in C++**

- **#include <iostream>**
- Lines beginning with a hash sign (#) are directives for the preprocessor.
- They are not regular code lines with expressions but indications for the compiler's preprocessor.
- This specific file (iostream) includes the declarations of the basic standard input-output library in C++

int main ()

- It does not matter whether there are other functions with other names defined before or after it
- The instructions contained within this function's definition will always be the first ones to be executed in any C++ program.

cout<< "Hello World";

- cout is declared in the iostream standard file within the std namespace
- so that's why we needed to include that specific file and to declare that we were going to use this specific **namespace** earlier in our code.
- Notice that the statement ends with a semicolon character (;).

- **return 0;**
- The return statement causes the main function to finish.
- Return may be followed by a return code (in our example is followed by the return code 0).

I/P

```
// my second program in C++
#include <iostream>
using namespace std;
int main ()
{
  cout << "Hello World! ";
  cout << "I'm a C++ program";
  return 0;
}
```

Hello World! I'm a C++ program

- **Ex/write a program to print the following message on the screen .**
- C++ is a programming language
- C++,based on the C programming

- ***#include <iostream.h>***
- ***Void* main ()**
- **{**
- **cout << " C++ is a programming language\n";**
- **cout << " C++ based on the C programming ";**
- **}**

- O/P
- C++ is a programming language
- C++ based on the C programming

- \n
- This character means newline on the screen.

- #include < iostream.h>
- main ()
- }
- cout<<“smoking is dangerous \n”;
- return0;
- {

- #include <iostream.h>
- main ()
- }
- cout<<“ smoking is dangerous”<<endl;
- return0;
- {

```
#include <
iostream.h>
Int main ()
{
cout<<“matrix”;
cout<<“matrix \n”;
cout<<“matrix \n\n”;
cout<<“matrix \n\n\n
”;
cout<<“matrix”;
return0;
```

- matrixmatr
  ix
- matrix
-
- Matrix
-
-
- Matrix